

“Movie Recommendation System using Upper Confidence Bound (UCB) Optimization”

by ushodaya namala

Submission date: 02-Apr-2026 03:40PM (UTC+0530)

Submission ID: 2920607823

File name: NOP_Report_pdf-1.pdf (1.17M)

Word count: 4327

Character count: 24376



¹
School of Computer Science and Engineering

JAIN (DEEMED -TO - BE) UNIVERSITY

B. Tech CSE-AIML

PROJECT REPORT ON

**“Movie Recommendation System using Upper Confidence Bound
(UCB) Optimization”**

³
Submitted in partial fulfilment for the award of the degree

Bachelor in Technology

COMPUTER SCIENCE AND AIML

Submitted by

NAME	USN
Swathi Priya R	23BTRCL084
Namala Ushodaya	23BTRCL225

Submitted to Dr.Sujeet S Japtap

Assistant professor

Faculty of Engineering&Technology

JAIN(Deemed-to-be-university)

BONAFIDE CERTIFICATE

Certified that this Mini Project report “Movie Recommendation System using Upper Confidence Bound (UCB) Optimization” is the bonafide work of Swathi Priya(23BTRCL084), Namala Ushodaya(23BTRCL225), who carried out the mini project work under my supervision.

SUPERVISOR

Er. MUKESH K RANJAN

2
Dept of Computer Science and
Engineering, AIML

JAIN DEEMED-TO-BE
UNIVERSITY

HEAD OF THE DEPARTMENT

Dr.CHANDRASEKAR V

2
Dept of Computer Science
and Engineering, AIML

JAIN DEEMED-TO-BE
UNIVERSITY

DATE OF THE VIVA VOCE EXAMINATION:

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to our project guide **Dr. Sujeet S Japtap** Assistant Professor, School of Computer Science and Engineering, Jain (Deemed-to-be) University, for her valuable guidance, encouragement, and continuous support throughout the development of this project titled “**Movie Recommendation System using Upper Confidence Bound (UCB) Optimization**”. I would also like to thank the **Head of the Department Dr. Chandrasekar V** for providing the opportunity and resources required to complete this project successfully. Finally, I express my gratitude to my family and friends for their motivation and support during the completion of this project.

Abstract

Recommender systems are of significant value in modern digital systems and have become an essential tool for assisting users in finding relevant information from a large dataset. One of the biggest challenges for a recommendation system is the trade-off between exploration and exploitation. This project aims to develop a movie recommendation system based on the Upper Confidence Bound (UCB) optimization algorithm. The UCB optimization algorithm is a popular optimization method for solving multi-armed bandit optimization problems. The project will use the MovieLens 100K dataset developed by GroupLens Research. The dataset includes 100,000 ratings from 943 users on 1,682 movies. The project will use the ratings data to develop a movie recommendation system based on the UCB optimization algorithm. The ratings will be mapped to a reward-based optimization problem where a high rating is considered a positive reward for a movie. The UCB optimization algorithm will be used to develop a dynamic recommendation algorithm for selecting movies for recommendation based on the average rewards and uncertainty of the movies' selection frequency. The algorithm will be simulated for multiple rounds of recommendation to find the movies that give a high reward and improve the quality of the recommendation over time. The experimental results will be evaluated based on metrics such as cumulative rewards and regret. The results will also be visualized in the form of a graph and an HTML-based dashboard for better visualization of the results. The results of the project will prove that the UCB optimization algorithm is a good optimization method for a recommendation system and will be able to find the movies that are most preferred by the user while also exploring other movies that are not so frequently recommended. The project will also prove that numerical optimization methods are useful for developing intelligent systems and making better decisions.

TABLE OF CONTENTS

Section	Title	Page No
	Abstract	4
Chapter 1	Introduction to Recommendation Systems	8-9
1.1	Background of Recommendation Systems	8
1.2	Evolution of Recommender Algorithms	8
1.3	Problem Statement	8
1.4	Objectives of the Project	9
1.5	Scope of the Work	9
Chapter 2	Literature Review and Related Work	10-12
2.1	Overview of Recommender Systems	10
2.2	Collaborative Filtering Approaches	10
2.3	Multi-Armed Bandit Algorithms	11
2.4	Upper Confidence Bound Optimization	12
Chapter 3	System Design and Dataset Description	13-15
3.1	System Architecture	13
3.2	Dataset Collection and Description	13
3.3	Data Preprocessing	14
3.4	Data Representation and Reward Mapping	14
3.5	Workflow Diagram	15
Section	Title	Page No
3.6	Hardware Requirements	15
3.7	Software Requirements	15
Chapter 4	Optimization Methodology	16-19
4.1	Introduction to Multi-Armed Bandit Problem	16
4.2	Upper Confidence Bound (UCB) Algorithm	16
4.3	Mathematical Formulation	17

4.4 Implementation of the Algorithm	17
4.5 Training and Simulation Process	18
4.6 Evaluation Metrics	19
Chapter 5 Experimental Results and Analysis	20–24
5.1 Experimental Setup	20
5.2 Cumulative Reward Analysis	20
5.3 Regret Curve Evaluation	21
5.4 Movie Recommendation Frequency	22
5.5 Top Recommended Movies	22
5.6 Performance Discussion	24
Chapter 6 Conclusion and Future Work	25–26
6.1 Summary of Findings	25
6.2 Future Scope	26
Chapter 7 References	27

List of figures

Figures	Name	Page no
1.	Worklow Diagram	14
2.	Cumulative Reward vs. Time	21
3.	Regret Curve vs. Time	22
4.	Movie Recommendation Frequency	23

CHAPTER 1: INTRODUCTION

1.1 Background of Recommendation Systems

In recent times, due to the rapid development of online platforms such as video streaming services, online shopping websites, and online media platforms, users are faced with a large volume of information and content. Recommendation systems have been developed to efficiently assist users in finding relevant information and content. Recommendation systems are information filtering systems that recommend items to a user based on the analysis of the user's behavior and interests.

Recommendation systems have been extensively used for recommending movies, products, music, and news articles. The traditional methods of recommendation systems include collaborative filtering methods, content-based filtering methods, and hybrid methods of recommendation systems. However, these methods are faced with several challenges such as cold start problems and inefficiency in exploring new items.

In order to overcome these challenges, optimization-based methods such as the multi-armed bandit optimization algorithm have been developed. The optimization algorithm is efficient in exploring new items and recommending items based on previous experiences.

1.2 Problem Statement

The main problem of a recommendation system is to select the most appropriate item for a user based on the limited information provided by the user. Recommendation systems that are based on recommending the most popular items may not yield the best results. Recommendation systems that are based on exploring new items may not yield the best results due to the low satisfaction of the user.

The main problem of this project is to develop a recommendation mechanism that balances the exploration and exploitation of items and maximizes user satisfaction. The project will use the optimization algorithm to recommend movies based on user feedback.

1.3 Objectives of the Project

The objectives of this project are:

- Design and implement a movie recommendation system by following an optimization-based approach.
- Apply the Upper Confidence Bound (UCB) algorithm for deriving optimal movie recommendations.
- Make use of the MovieLens data for training the movie recommendation model.
- Evaluate the performance of the UCB-based movie recommendation algorithm.
- Visualize the movie recommendation results by plotting graphs.

1.4 Scope of the Project

The scope of this project revolves around the development of a simulation-based movie recommendation system by considering the MovieLens data. The movie recommendation system analyzes the movie ratings data and uses the UCB algorithm for deriving movie recommendations for different rounds of movie recommendations.

The major emphasis of this project is on the implementation of the UCB-based movie recommendation system and evaluating the performance of the UCB-based movie recommendation algorithm. However, the approach can be easily applied to other movie recommendation scenarios.

1.5 Significance of the Study

The movie recommendation system can efficiently utilize the numerical optimization approach for deriving movie recommendations. The UCB-based movie recommendation system can efficiently learn the user preferences by exploring new movie recommendation possibilities.

The movie recommendation system can efficiently utilize the UCB-based approach for deriving movie recommendations. The movie recommendation system can efficiently utilize the UCB-based approach for deriving movie recommendations. The movie recommendation system can efficiently utilize the UCB-based approach for deriving movie recommendations.

CHAPTER 2: LITERATURE SURVEY

2.1 Overview of Literature

Recommendation systems are now a vital component of modern digital platforms, such as online shopping websites, movie streaming services, and music platforms. The key objective of these systems is to provide a prediction of user preferences and recommend items that are most likely to be relevant to a particular individual. Over the years, various techniques have been developed to enhance the accuracy and efficiency of these systems.

Traditionally, various rule-based techniques and simple statistical models were used in these systems. However, with the increase in data size and complexity, machine learning techniques were integrated to enhance their accuracy and efficiency. The techniques include collaborative filtering, content-based filtering, and hybrid models of recommendation systems.

2.2 Survey of Existing Research

Collaborative filtering is a widely used technique in recommendation systems, which relies on analyzing similarities between users or items based on their interactions in the system. User-based collaborative filtering recommends items to a user based on similar preferences from other users, while item-based collaborative filtering recommends items similar to those liked by a particular individual in the past.

Matrix factorization techniques were later integrated to enhance the accuracy of these systems by reducing a large matrix to a lower-dimensional latent factor space, which captures hidden relationships between users and items, thereby improving prediction accuracy. However, these traditional techniques face a major challenge in balancing exploration and exploitation in these systems. To overcome this, researchers integrated a new class of algorithms, referred to as multi-armed bandit algorithms, which dynamically learn user preferences by exploring new options while exploiting high-performing options.

The Upper Confidence Bound (UCB) is a widely used bandit algorithm in these systems, which selects items based on their historical reward and associated uncertainty with these items, ensuring a balance between exploration and exploitation.

2.3 Comparative Analysis of Techniques

Technique	Advantages	Limitations
Content-Based Filtering	Personalized recommendations	Limited diversity
Collaborative Filtering	Utilizes user similarity	Cold start problem
Matrix Factorization	High prediction accuracy	Computational complexity
Multi-Armed Bandit	Balances exploration and exploitation	Requires iterative learning
UCB Algorithm	Efficient decision making	Assumes stable reward distribution

2.4 Summary of Literature

From the literature survey, it is clear that the recommendation system has made significant improvements by incorporating machine learning and optimization techniques. The traditional approach of collaborative filtering has shown impressive performance. However, it has some drawbacks in dynamic environments.

The multi-armed bandit algorithms, specifically the UCB algorithm, have shown promising results in improving the performance of the recommendation system by continuously learning from the user interactions. Thus, in this project, the UCB algorithm is employed to develop a more efficient movie recommendation system.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

The system architecture illustrates the structure of the movie recommendation system. The system architecture comprises multiple modules that collaborate to process the data, apply the optimization algorithm, and provide movie recommendations.

The major components of the movie recommendation system include:

1. **Dataset Input Module:** This module will be used to input the MovieLens dataset containing ratings from numerous users.
2. **Data Preprocessing Module:** This module will be used to clean and preprocess the input dataset to a form that can be used in the movie recommendation system.
3. **Recommendation Engine:** This module will be used to apply the Upper Confidence Bound algorithm to recommend movies to the end-user based on reward statistics.
4. **Evaluation Module:** This module will be used to assess the performance of the movie recommendation algorithm.
5. **Visualization Module:** This module will be used to display graphs and charts to illustrate the performance of the movie recommendation algorithm.

The movie recommendation system will process the ratings from end-users and simulate multiple rounds of movie recommendations to learn which movies have higher reward values over time.

3.2 Dataset Collection

The dataset used in this project is the MovieLens 100K dataset, which is one of the most common datasets used in movie recommendation system projects. The dataset was created by GroupLens Research, a research laboratory in the Department of Computer Science and Engineering at the University of Minnesota.

The major features of the dataset include:

- 100,000 movie ratings
- 943 users
- 1,682 movies
- Rating scale: 1 to 5

Each records contains the following attributes with Description:

Attribute	Description
User ID	Unique identifier of the user
Movie ID	Unique identifier of the movie
Rating	User rating (1–5)
Timestamp	Time when the rating was provided

This dataset will give a realistic environment for testing and evaluating a recommendation algorithm.

3.3 Data Preprocessing

Data preprocessing is a vital step in preparing a dataset before implementing a particular algorithm. The dataset contains ratings, which are in a raw format and must be converted to a reward format before implementing the UCB algorithm.

The data preprocessing steps are:

1. Loading a dataset using Python libraries such as Pandas.
2. Dealing with missing data in a dataset and checking its consistency.
3. Converting ratings to a reward format for implementing a bandit algorithm.
4. Preparing a matrix showing interactions between users and movies.
5. Normalizing and preparing a dataset for simulation.

In this project, ratings equal to 4 and above are taken as a positive rating, and a reward value of 1 is assigned, while ratings below 4 are assigned a reward value of 0.

3.4 Workflow Diagram

The workflow diagram of this system represents a series of steps required in recommending a movie to a particular user.

The steps are:

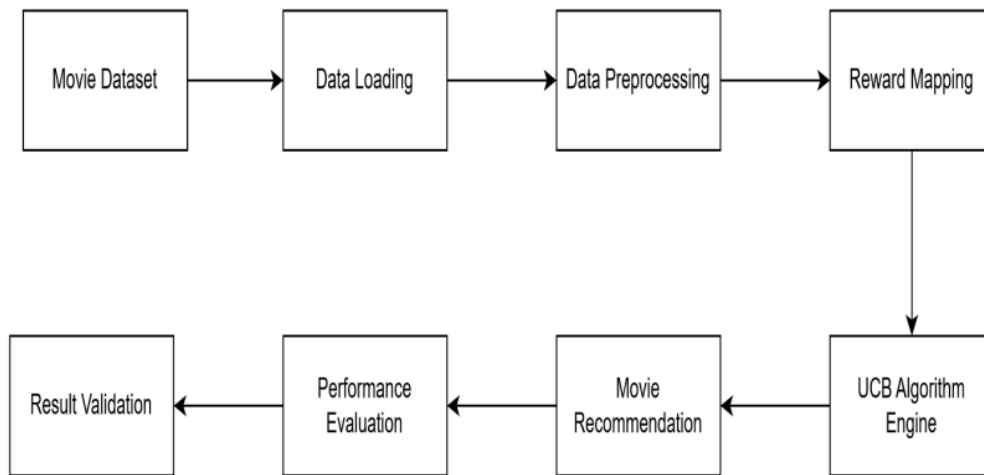


Figure no. 1 – Workflow Diagram

This workflow guarantees the efficiency of the system in processing the data and making relevant recommendations.

1. Dataset loading
2. Data preprocessing
3. Reward matrix creation
4. UCB algorithm execution
5. Movie recommendation selection
6. Performance evaluation
7. Visualization of results

3.5 Hardware Requirements

The hardware requirements for the project are low because the project is based on the simulation of the algorithm. It is not based on deep learning.

Minimum hardware requirements for the project:

- Processor: Intel Core i5 and above
- RAM: 8 GB RAM
- Disk space: 10 GB free disk space
- Operating System: Windows, Linux, macOS, etc.

These requirements are sufficient to execute the optimization algorithm.

3.6 Software Requirements

The project is developed using the Python programming language because it is the best language to support machine learning and data analysis.

Software requirements for the project:

- Python 3.8 or above
- NumPy library for numerical computations
- Pandas library for data manipulation
- Matplotlib library for data visualization

Development tools used for the project:

- Visual Studio Code for coding purposes
- Jupyter Notebook for visualization purposes

CHAPTER 4: METHODOLOGY

4.1 Dataset Description

The dataset used in this project is the MovieLens 100K dataset. It is one of the popular datasets used for the development of the recommendation system. It contains 100,000 ratings from 943 users on 1,682 movies. Each rating is ranging from 1 to 5, which represents the user's preference for the movie.

Attribute	Description
User ID	Unique identifier of the user
Movie ID	Unique identifier of the movie
Rating	Rating given by the user (1–5)
Timestamp	Time when the rating was given

For the purpose of this project, the dataset is converted into a reward-based form that is required for implementing the multi-armed bandit algorithm.

4.1 Data Preprocessing

Before the optimization algorithm is applied to the dataset, it needs to be preprocessed and cleaned for better results. The data preprocessing stage is important for preparing the dataset for the recommendation algorithm.

The data preprocessing involves the following steps:

1. Loading the dataset into the Python environment using Pandas.
2. Removing any columns that are not required for the project.
3. Organizing the data into a user-movie interaction matrix.
4. Changing the ratings into rewards.

In the project, ratings are converted into rewards based on the following conditions:

Rating ≥ 4 -> Reward = 1

Rating < 4 -> Reward = 0

4.3 Multi-Armed Bandit Problem

The problem of recommending movies to a user is analogous to the Multi-Armed Bandit problem. In the problem, each movie is considered an 'arm' of the bandit. When a user selects a movie and provides a positive rating for it, a reward is provided to the bandit algorithm.

The purpose of the bandit algorithm is to maximize the rewards over a period of time by exploring which movies are providing better rewards.

However, the problem of such systems is finding the right balance between the following strategies:

Exploration - Selecting new movies that have not been recommended before.

Exploitation - Selecting movies that have provided better rewards in the past.

4.4 Upper Confidence Bound (UCB) Algorithm

The Upper Confidence Bound algorithm is a method of finding a solution for the exploration and exploitation problem in the context of the multi-armed bandit problem. The algorithm is based on selecting movies based on the average rewards and the uncertainty of the movies.

The mathematical formulation of the UCB algorithm is provided by:

$$UCB_i = \bar{x}_i + \sqrt{\frac{2 \ln n}{n_i}}$$

Where:

- $\bar{x}_i$ = average reward obtained from movie i
- n = total number of recommendation rounds
- n_i = number of times movie i has been selected

The above equation consists of two terms. The first term is the exploitation part, while the second part is the exploration part. The algorithm combines the exploitation part with the exploration part to ensure that both new movies and high-performing movies are recommended.

4.5 Training and Simulation Process

The recommendation system is trained through the simulation process. The algorithm selects movies repeatedly through multiple rounds. In every round, the system selects the movie with the best UCB score.

The process followed during the training process is as follows:

1. Initialization: The system initializes the number of selections for every movie as well as the reward associated with every movie.
2. For every recommendation round:
 - o Compute the UCB score for every movie.
 - o Select the movie with the best UCB score.
 - o Observe the reward from the data set.
 - o Keep track of the reward.

3. Repeat the above process for many rounds, i.e., 10,000 or 100,000.

As the algorithm processes many rounds, the algorithm is able to determine the movies with high reward potential and selects the movies accordingly.

4.6 Evaluation Metrics

In order to assess the performance of the recommendation system, the algorithm follows the following metrics:

Cumulative Reward

The cumulative reward is the reward earned from the entire recommendation rounds. The higher the reward, the more efficient the recommendation system is.

Regret

Regret is the difference between the reward earned by the algorithm and the maximum reward possible. The lower the regret, the more efficient the recommendation strategy is.

Movie Selection Frequency

The above metric measures the frequency with which the algorithm selects the movies. The movies with high reward potential are the ones that are often recommended by the algorithm.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Experimental Setup

The experimental setup was conducted by implementing the algorithm in Python. Libraries used include NumPy, Pandas, and Matplotlib. The MovieLens 100K dataset was used to simulate the rounds of movie recommendations. The Upper Confidence Bound algorithm was used to find out which movie should be recommended in each round.

The simulation was conducted for 100,000 rounds of movie recommendations. In each round, the algorithm calculated the Upper Confidence Bound value for all movies and recommended the movie with the maximum confidence bound. Depending on the reward generated in the dataset, the system updated the reward values.

The evaluation metrics used to analyze the results include cumulative reward, regret curve, and movie recommendation frequency.

5.2 Cumulative Reward Analysis

The cumulative reward curve indicates the total reward generated by the algorithm in multiple rounds of movie recommendations. The cumulative reward increases steadily as the number of rounds increases. This indicates that the algorithm learns which movies generate higher rewards. Initially, the system explores movies to learn about their performance. As the algorithm learns more about the movies, it starts to exploit the movies that generate higher rewards. This leads to an increase in cumulative reward over time. The results indicate that the UCB algorithm improves

the quality of movie recommendations over time by learning from the feedback.

Cumulative Reward vs. Time

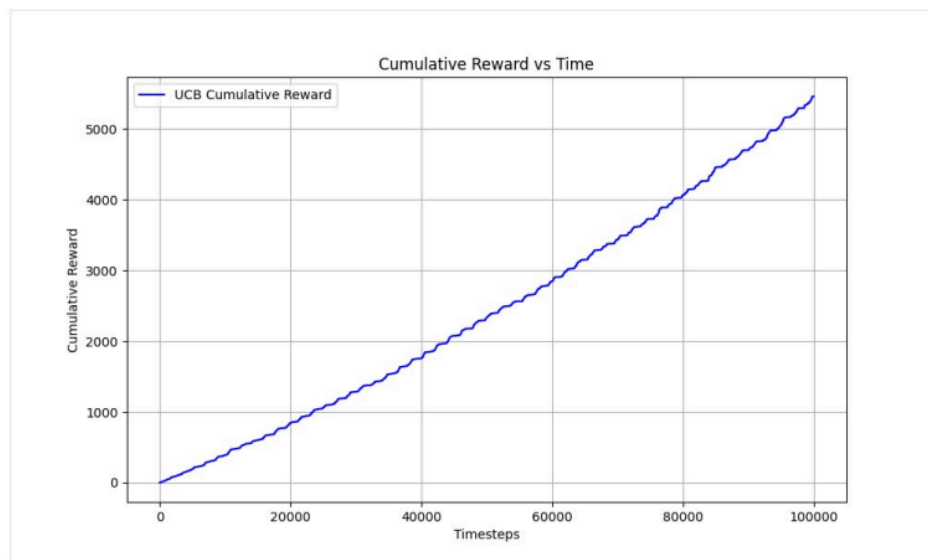


Figure no: 2 – Cumulative Reward vs. Time Explanation:

- Indicates how the algorithm enhances recommendations over time.

5.3 Regret Curve Evaluation

Regret curve is a measure of how the difference is obtained by subtracting the reward achieved by the algorithm from the maximum possible reward that could have been achieved by selecting the best movie at every round.

Regret increases rapidly at the beginning of the experiment as the algorithm is exploring different movies; however, as more rounds are reached in the experiment, the rate at which regret increases declines as the

system is able to learn what movies offer a high reward.
Regret Curve vs. Time

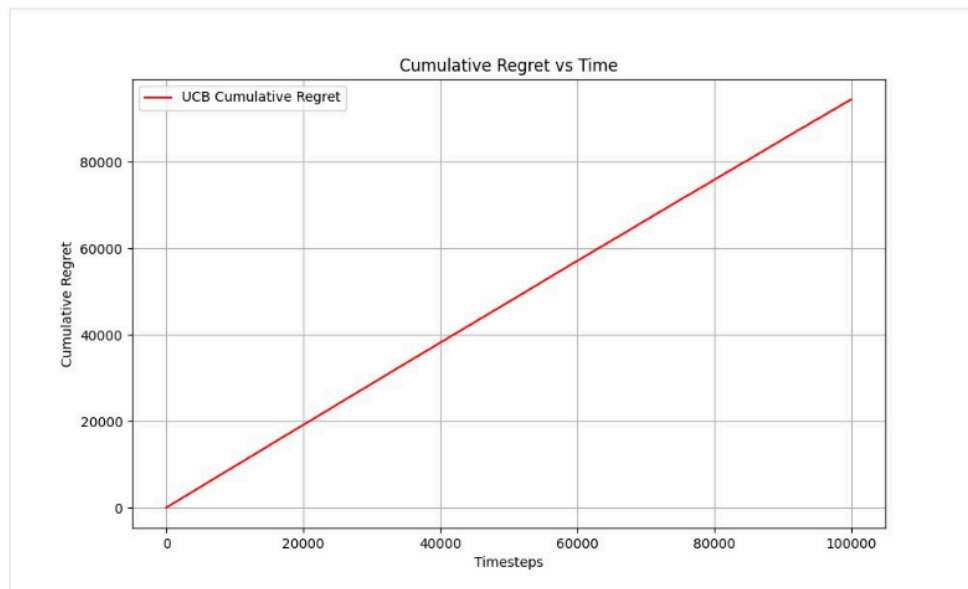


Figure no : 3 – Regret Curve vs. Time Explanation:

- Displays exploration vs. exploitation performance.

5.4 Movie Recommendation Frequency

The movie recommendation frequency graph illustrates the frequency of each movie being recommended in the simulation. The more frequently a movie is recommended, the higher the reward it generates.

In the experiment, it has been shown that a few movies have been recommended more frequently than others. This is a demonstration of the exploitation feature of the UCB algorithm. The movies that perform better are recommended more frequently. At the same time, other movies that perform less frequently are not

ignored

Movie Recommendation Frequency

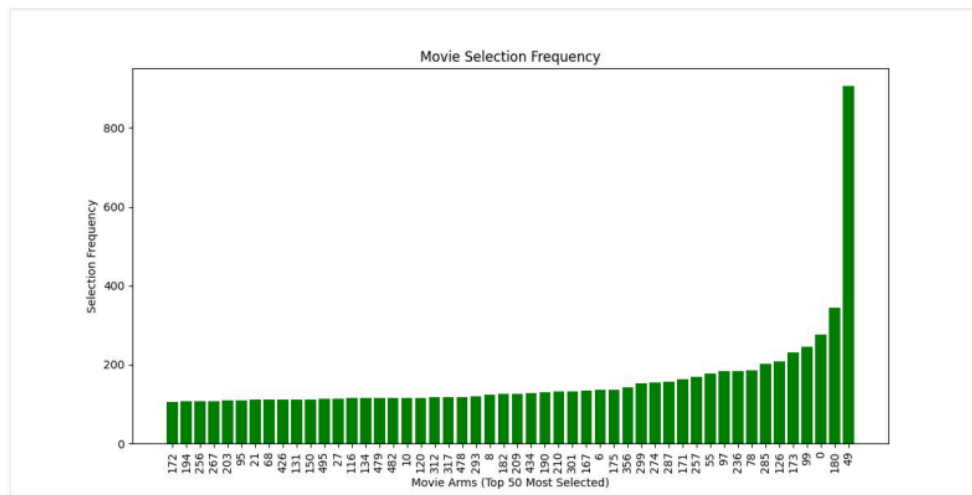


Figure no : 4 - Movie Recommendation Frequency Distribution Explanation:

- It shows which movies were recommended the most.

5.5 Top Recommended Movies

This system recognizes the movies that frequently receive higher rewards during the recommendation process. These movies can be considered the most preferred by the users based on the data set.

Some examples of the frequently recommended movies can be the popular ones like this:

- Star Wars
- Raiders of the Lost Ark
- The Godfather
- Toy Story

- Return of the Jedi

Section 5: Top Recommended Movies

- Star Wars (1977) (ID: 50) - Selected 906.0 times, Avg Reward: 0.5121
- Return of the Jedi (1983) (ID: 181) - Selected 344.0 times, Avg Reward: 0.4128
- Toy Story (1995) (ID: 1) - Selected 275.0 times, Avg Reward: 0.3818
- Fargo (1996) (ID: 100) - Selected 246.0 times, Avg Reward: 0.3659
- Raiders of the Lost Ark (1981) (ID: 174) - Selected 231.0 times, Avg Reward: 0.3550
- Godfather, The (1972) (ID: 127) - Selected 207.0 times, Avg Reward: 0.3382
- English Patient, The (1996) (ID: 286) - Selected 201.0 times, Avg Reward: 0.3333
- Fugitive, The (1993) (ID: 79) - Selected 185.0 times, Avg Reward: 0.3189
- Silence of the Lambs, The (1991) (ID: 98) - Selected 183.0 times, Avg Reward: 0.3169
- Jerry Maguire (1996) (ID: 237) - Selected 183.0 times, Avg Reward: 0.3169

Figure : 4 – Top Recommendation Movies

5.6 Performance Discussion

The experimental results indicate that the UCB algorithm performs well in the movie recommendation problem. The algorithm can effectively trade off exploration and exploitation, and gradually learn the performance of the movies.

The observations from the experimental results are:

- The cumulative reward curve increases over time.
- The regret curve increases but at a slower rate. This means that the algorithm learns the optimal decision.
- A few movies have higher frequency in the recommendations because they have higher rewards. This demonstrates the efficiency of optimization-based algorithms in improving the performance of movie recommendation systems.

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Conclusion

This project has implemented a movie recommendation system based on the optimization algorithm, UCB. The algorithm is widely used in multi-armed bandit problems. The movie recommendation system can provide dynamic recommendations to the end-user. It can efficiently trade off exploration and exploitation. The algorithm can learn the performance of movies over time. The movie recommendation system can provide efficient recommendations to the end-user. The project has used the MovieLens 100K dataset to simulate 100,000 rounds of recommendations. The algorithm has calculated the confidence bounds of movies and chosen the movie with the highest upper confidence value. The experimental results have shown that the algorithm can learn the performance of movies over time. The cumulative reward curve increases over time. The experimental results have shown that the algorithm can efficiently trade off exploration and exploitation. The regret curve increases but at a slower rate. This means that the algorithm learns the optimal decision. This demonstrates that the performance of the movie recommendation system improves over time.

Another thing that can be observed from the results is that some movies were recommended more often than others. This means that the system has successfully identified the movies that have higher reward values and has given more priority to those movies during the recommendation.

Overall, the implementation has shown how optimization techniques can be used in recommendation systems for more efficient decisions.

6.2 Future Scope

The system proposed in this paper has successfully shown how UCB optimization can be used in movie recommendation. However, there are several improvements that can be made in future.

One of the improvements that can be made in future is to use more complex reward functions instead of the simple reward function that has been used in this paper. For example, context-aware movie recommendations can be made.

Another improvement that can be made in future is to use more complex algorithms such as the contextual bandit problem for more accurate results.

Another improvement that can be made in future is to use deep learning-based algorithms such as neural collaborative filtering for more accurate results.

Another possible improvement that can be made in future is to use this recommendation engine in a real-time environment such as a web application. In this case, the users can use the system for movie recommendations.

Another possible improvement that can be made in future is to use this approach in other fields such as music recommendation, advertising, product recommendation in e-commerce websites, news feeds, etc.

REFERENCES

1. Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," *Machine Learning Journal*, vol. 47, no. 2–3, pp. 235–256, 2002.
2. Lihong Li, Wei Chu, John Langford, and Robert E. Schapire, "A Contextual-Bandit Approach to Personalized News Article Recommendation," *Proceedings of the 19th International World Wide Web Conference*, 2010.
3. F. Maxwell Harper and Joseph A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, 2015.
4. GroupLens Research, "MovieLens 100K Dataset," University of Minnesota.
5. Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
6. Charu C. Aggarwal, *Recommender Systems: The Textbook*, Springer, 2016.
7. Scikit-learn Developers, "Machine Learning in Python"
8. Python Software Foundation, "Python Programming Language Documentation"

GitHub Link : <https://github.com/SwathiPriya37/ucb-movie-recommendationbandit.git>

Zip File :



!"#\$%&'()*\$*)"&%%)+,-.(&+\$#-+,(./0(1

"Movie Recommendation System using Upper Confidence Bound (UCB) Optimization"

ORIGINALITY REPORT

2%	2%	1%	1%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	andhrauniversity.edu.in	1%
	Internet Source	
2	Submitted to City University of Seattle	1%
	Student Paper	
3	rjpn.org	1%
	Internet Source	

Exclude quotes	On	Exclude matches	< 1%
Exclude bibliography	On		